

First, reproduce learning in a simpler setting.

A recursive language model (RLM) can use code and ask helper models to solve parts of a problem. We eventually want to train those choices.

Today's question

Can we use a published training recipe to improve a model's math answers on questions excluded from training?

We use the authors' code for **Dr. GRPO**, a reinforcement learning (RL) method. It learns from scores assigned to the model's own answers.

First check: The authors' released model scored 73.2% in our test, versus 74.2% reported. That checks their saved model, not our own training.

Liu et al., *Understanding R1-Zero-Like Training: A Critical Perspective* (2025).

Official code; our run is much smaller than the paper's experiments.

Reward correct attempts, then change the model.

Illustrative question: Solve $3x + 2 = 14$. The correct answer is $x = 4$.

The model's attempts	Checker score	Learning direction
Two answers end in $x = 4$.	1 each	Make these more likely.
Two end in $x = 5$ or $x = 6$.	0 each	Make these less likely.

Each attempt is compared with the average score for that question. These differences guide changes to the model's numerical settings, or **weights**.

The checker judges the final answer. We do not supply worked solutions for the model to copy.

Simplified illustration with four attempts; our runs use eight. If every attempt receives the same score, this question supplies no relative learning signal.

We train on math, then test on different questions.

We add RL to the pretrained math model **Qwen2.5-Math-1.5B**.

Training budget	Questions	Weight updates
Completed short runs	512	32
Longer run now underway	4,080	255 planned

We test on **MATH500: 500 math questions excluded from our weight training**. A fixed subset of 64 tracks progress.

Two ways of asking the same question:

Chat style: A conversation with instructions to explain the steps and mark the final answer.

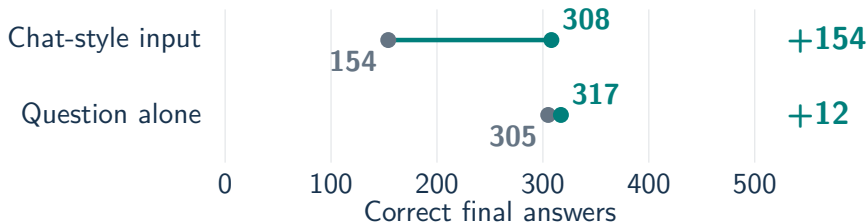
Question alone: Give only the original question.

Every run starts from the original model; one A100 GPU, eight answers per training question. The 64 monitored questions are not an untouched final test.

The input format changes the size of the gain.

Correct answers out of the same 500 MATH500 questions.

Gray: starting model. Teal: after short chat-style training.

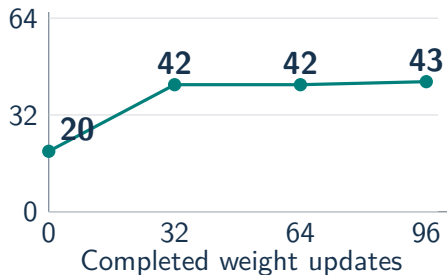


Lesson: The starting model already solves many more questions with a better prompt. Training must improve beyond that stronger starting point.

Separate question-only training runs added 9 and 1 correct answers, versus the same baseline of 305. No reliable gain established yet. Full comparison table is in the guide.

Longer training: an early gain, then little change.

Correct final answers out of the same 64 monitoring questions.



One training run, with the same questions, chat-style input, checker, and answer-length limit.

At the last check, five answers improved and four became wrong. The total rose by just one.

Next: Finish this run. Then use short runs to test larger learning steps and two learning passes per batch, with 128-question checks.

Interim result from one ongoing run, not a final benchmark score. We have not yet shown that this longer recipe beats the stronger starting setup or the short runs.